UTN FRLP

Desarrollo de Software 2026

Introducción a React

Clase #08 - 18 de mayo del 2026

Matías Corti - Martina García Améndola

Temas

- Entender qué es React y por qué se usa
- Conocer los 4 conceptos clave: **JSX, Componentes, Props, Estado**
- Crear un proyecto React + TypeScript con Vite
- Escribir componentes con props tipadas y estado local
- Comenzar el frontend del TP grupal



¿POR QUÉ NECESITAMOS HERRAMIENTAS MODERNAS?

Problemas con Vanilla JavaScript

- Manipulación compleja del DOM
- Código repetitivo y difícil de mantener
- Organización difícil en proyectos grandes
- Reutilización limitada de código

```
// Por ejemplo: Para mostrar una lista de usuarios...
document.getElementById('user-list').innerHTML = '';
users.forEach(user => {
   const div = document.createElement('div');
   div.className = 'user-card';
   div.innerHTML =
`<h3>${user.name}</h3><p>${user.email}</p>`;
   document.getElementById('user-list').appendChild(div);
});
```



HERRAMIENTAS FRONTEND MODERNAS

Conjunto de librerías y frameworks que facilitan el desarrollo de aplicaciones web

¿Qué nos dan?

- 🔧 Estructura organizada del código
- 🔄 Reutilización de componentes
- ⚡ Mejor rendimiento
- 🛠️ Herramientas de desarrollo
- 👥 Comunidad y documentación

Ejemplos populares:
React *(librería)* | Vue.js | Angular *(framework)*



¿POR QUÉ REACT?

Creado por Facebook (Meta) en 2013

Usado por: Netflix, Airbnb, Instagram, WhatsApp Web, Mercado Libre

🏆 **Ventajas clave**

- Componentes reutilizables
- Virtual DOM = mejor rendimiento
- Curva de aprendizaje suave
- Comunidad masiva
- Alta demanda laboral:
  📊 *En Argentina, ~70% de las ofertas de frontend piden React*



4 CONCEPTOS CLAVE DE REACT

⚛️ JSX

🧩 COMPONENTES

🎁 PROPS

📊 ESTADO

💡 Dominando estos 4 conceptos, ya pueden crear aplicaciones increíbles

⏭️ Vamos uno por uno...



JSX

JS + HTML

¿Qué es JSX?

JavaScript que parece HTML, pero NO es HTML

- Puedo poner expresiones JS entre {}
- Atributos van en camelCase: className, no class
- JSX hace que nuestros componentes sean dinámicos, no estáticos

```jsx
function TarjetaProducto() {
  const producto = "Smartphone";
  const precio = 899;
  const disponible = true;

  return (
    <div className="producto">
      <h2>{producto}</h2>                          {/* Variable JS */}
      <p>${precio}</p>                             {/* Número JS */}
      <p>Descuento: ${precio * 0.9}</p>            {/* Operación JS */}
      <p>Stock: {disponible ? "✅" : "❌"}</p>     {/* Condición JS */}
    </div>
  );
}
```

COMPONENTES

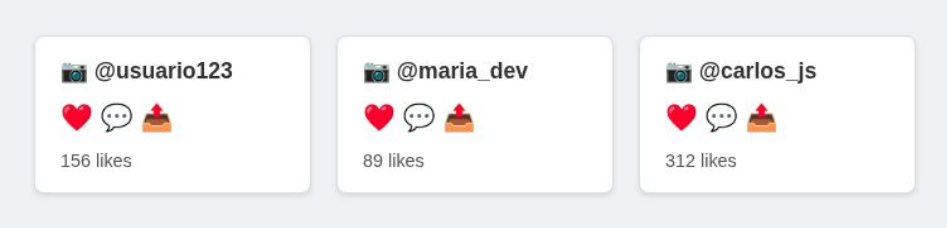
🔄 **1 componente**

⬇

∞ usos con datos diferentes

¿Qué son?

Piezas reutilizables de interfaz de usuario



↑ Mismo componente "PostCard" usado 3 veces

```jsx
function PostCard(props) {
  return (
    <div className="post">
      <h3>📷 {props.usuario}</h3>
      <div>❤️ 💬 📤</div>
      <p>{props.likes} likes</p>
    </div>
  );
}
```

- **1 componente <=> 1 función que retorna JSX**
- El nombre va en `PascalCase` (`LibroCard`, no `librocard`)
- Se usa como una etiqueta HTML: `<PostCard />`
- Tiene que retornar **UN** solo elemento padre (o un Fragment `<> </>`)



PROPS

🎯 **1 molde + diferentes datos**

⬇

infinitas variaciones

¿Qué son?

Argumentos que se le pasan a un componente

```jsx
function PeliculaCard(props) {
  return (
    <div className="movie-card">
      <img src={props.imagen} />
      <h3>{props.titulo}</h3>
      <p>⭐ {props.rating}/5</p>
      <p>{props.año} • {props.genero}</p>
    </div>
  );
}
```

```
// Un componente, tres películas diferentes
<PeliculaCard titulo="Avengers" rating={5} año={2019} genero="Acción" />
<PeliculaCard titulo="Stranger Things" rating={5} año={2022} genero="Sci-fi" />
<PeliculaCard titulo="Narcos" rating={4} año={2015} genero="Drama" />
```

🎁 Props tipadas con TypeScript

jsx

```jsx
function LibroCard(props) {
  return <h3>{props.titulo}</h3>;        // ¿props.titulo es string? ¿number?
}                                        // ¿Existe? Te enterás en runtime.
```

tsx

```tsx
type LibroCardProps = {
  titulo: string;
  autor: string;
  precio: number;
};

function LibroCard({ titulo, autor, precio }: LibroCardProps) {
  return <h3>{titulo}</h3>;
}
```



ESTADO

📊 **Estado = memoria del componente**

💡 **Props vienen de afuera**

¿Qué es?

Información que puede cambiar y hace que el componente se actualice

```jsx
// ANTES (sin estado): botón "muerto"
<button>❤️ Like</button>  {/* Siempre igual */}
```

```tsx
// DESPUÉS (con estado): botón "vivo"
import { useState } from 'react';

function BotonLike() {
  const [likes, setLikes] = useState<number>(0);
  const liked = likes > 0;

  return (
    <button onClick={() => setLikes(likes + 1)}>
      {liked ? "❤️" : "🤍"} {likes} likes
    </button>
 );
}
```

Click → Estado cambia → Pantalla se actualiza ¡AUTOMÁTICAMENTE!



ESTADO

Reglas de Estado

```tsx
// ❌ NUNCA: asignación directa
count = count + 1;    // NO HACE NADA
```

```tsx
// ✅ SIEMPRE: usar la función set
setCount(count + 1);  // OK
```

Por qué:

- Asignar directo **NO le "avisa"** a React que algo cambió
- React **solo re-renderiza** cuando llamamos a **setXxx(...)**
- Cuando estado cambia => React "re-dibuja" el componente

```tsx
// ⚠️ IMPORTANTE: siempre importar el Hook useState de React
import { useState } from 'react';
```



COMPARACIÓN

ANTES

javascript

```javascript
let count = 0;
document.getElementById('button').addEventListener('click', function() {
  count++;
  document.getElementById('counter').innerHTML = count;
});
```

DESPUÉS

jsx

```jsx
import { useState } from 'react';

function Counter() {
  // [valor actual, función para cambiarlo]
  const [count, setCount] = useState<number>(0);
  //                              ↑ tipo    ↑ valor inicial
  return (
    <button onClick={() => setCount(count + 1)}>{count}</button>
  );
}
```

🎯 Resultado:
¡El mismo comportamiento!

✨ Diferencia:
Menos código, más claro



Instalación de Entorno



PREPARANDO NUESTRO ENTORNO

```
VITE v8.0.13  ready in 318 ms

→  Local:   http://localhost:5173/
→  Network: use --host to expose
→  press h + enter to show help
```

[React.dev: Crear App]

Necesitamos:

1️⃣ **Node.js** (runtime de JavaScript)

2️⃣ **NPM** (gestor de paquetes)

3️⃣ **Vite** (herramienta oficial recomendada)

```bash
### Parado en el directorio donde irán los proyectos

# Crear la App con Vite, incluido TypeScript:
npm create vite@latest mi-proyecto -- --template react-ts

cd mi-proyecto      # Ingresar al directorio del proyecto
npm install         # Instalar las dependencias
npm run dev         # Iniciar el servidor local

# Acceder con el navegador a http://localhost:5173/
```



ESTRUCTURA DEL PROYECTO

```
mi-proyecto/
├── public/          ← Archivos estáticos
├── src/             ← Nuestro código
│   ├── App.tsx      ← Componente principal
│   ├── main.tsx     ← Punto de entrada
│   └── App.css      ← Estilos
├── index.html       ← Index HTML donde se inyecta la App React
├── node_modules/    ← Código fuente de dependencias instaladas
├── package.json     ← Dependencias configuradas
├── tsconfig.json    ← config de TypeScript
└── vite.config.ts   ← Código fuente de dependencias instaladas
```

🔍 **Archivos/Carpetas importantes:**

📂 **src/** Trabajamos casi todo en este directorio

📄 **App.tsx:** Aquí trabajaremos principalmente

📄 **main.tsx:** Conecta React con el DOM, tocamos poco

Y en el TP grupal...



```
ds2026-tp-grupo-XX/
├── documentacion/  ← esta carpeta ya la vienen trabajando
├── frontend/       ← acá adentro va Vite (estructura completa)
│   ├── src/
│   ├── package.json
│   └── ...
├── (backend/)      ← se sumará en el Bloque 3
└── README.md
```

Ahora agregamos solo **frontend/** con la estructura completa de los slides anteriores.

El lugar de **backend/** ya está reservado para sumar en el Bloque 3.



CREANDO NUESTRO PRIMER COMPONENTE

tsx

App.tsx

```tsx
function LibroCard() {
  return (
    <div className="card">
      <h3>El Aleph</h3>
      <p>Jorge Luis Borges</p>
      <p>$15.000</p>
    </div>
  );
}

function App() {
  return (
    <div>
      <h1>Mi Librería</h1>
      <LibroCard />
    </div>
  );
}

export default App;
```

🔑 **Puntos clave:**

- Los componentes son funciones que retornan JSX
- Nombres siempre en PascalCase
- Se usan como etiquetas HTML: `<MiComponente />`



PROPS: PASANDO DATOS A COMPONENTES

tsx

📄 App.tsx

```tsx
type LibroCardProps = {
  titulo: string;
  autor: string;
  precio: number;
};

function LibroCard({ titulo, autor, precio }: LibroCardProps) {
  return (
    <div className="card">
      <h3>{titulo}</h3>
      <p>{autor}</p>
      <p>${precio}</p>
    </div>
  );
}

function App() {
  return (
    <div>
      <h1>Mi Librería</h1>
      <LibroCard titulo="El Aleph" autor="Borges" precio={15000} />
      <LibroCard titulo="Rayuela" autor="Cortázar" precio={18000} />
      <LibroCard titulo="Ficciones" autor="Borges" precio={14000} />
    </div>
  );
}

export default App;
```



Incorporando Estado

tsx

App.tsx

```tsx
import { useState } from 'react';

type LibroCardProps = {
  titulo: string;
  autor: string;
  precio: number;
};

function LibroCard({ titulo, autor, precio }: LibroCardProps) {
  const [likes, setLikes] = useState<number>(0);

  return (
    <div className="card">
      <h3>{titulo}</h3>
      <p>{autor}</p>
      <p>${precio}</p>
      <button onClick={() => setLikes(likes + 1)}>
        ❤️ {likes}
      </button>
    </div>
  );
}

...    //  Mantenemos el resto como estaba
```

Modificar LibroCard, agregamos un botón de "Me gusta" en cada card para que cada libro tenga su propio contador.



Competencia



Competencia

Componente del Día

🏆 Componente del Día

Fase 1

🕐 **15 min** para crear un componente **original**

💡 Cualquier idea: lo que se les ocurra

📋 Debe tener **props tipadas** (al menos 2 props)

🎁 **Bonus:** sumar **useState** si llegan

🏅 Los tres más votado se lleva **1 punto extra en concepto**

Reglas individuales

- Cada uno en su compu, en un proyecto Vite
- Pueden mirar la doc, pueden usar internet
- Si se traban, consulten



Competencia

Componente del Día

🏆 **Componente del Día**

Fase 2

🕐 **10 minutos**

👥 **Junten grupos de 4 a 6 con los de al lado** (no los grupos del TP)

Cada Grupo

- Cada uno muestra su componente a los demás (1-2 min cada uno)
- Discuten cuál les parece **más original**
- Eligen **UNO** para representar al grupo
- Pueden sugerir mejoras puntuales
- El elegido sube su código en el hilo "**[AP-C08] Componente del día**" del canal **#actividades** de Discord, mencionando a los demás miembros del grupo

🏆

Competencia

Componente del Día

🏆 **Componente del Día**

Fase 3: 🗳️ Votación final

- Cada alumno vota TRES componentes
- ¡Elijan los mejores!



LIBRERÍAS DE UI PROFESIONALES



OPCIÓN 1 BOOTSTRAP REACT

[react-bootstrap]

Paso 1: Instalar

```bash
npm install bootstrap react-bootstrap
```

Paso 2: Importar estilos (`main.tsx`)

```tsx
import 'bootstrap/dist/css/bootstrap.min.css;
```

Componente de ejemplo

```tsx
import { Card, Button } from 'react-bootstrap';

function LibroCard({ titulo, autor, precio }: LibroCardProps) {
 return (
   <Card style={{ width: '18rem' }}>
     <Card.Img variant="top" src={imagen} />
     <Card.Body>
       <Card.Title>{titulo}</Card.Title>
       <Card.Text>{autor}</Card.Text>
       <Button variant="primary">Ver más</Button>
     </Card.Body>
   </Card>
 );
}
```



OPCIÓN 2 MATERIAL-UI (MUI)

[Material UI]

Paso 1: Instalar

```bash
npm install @mui/material @emotion/react @emotion/styled
npm install @mui/icons-material
```

Componente de Ejemplo

```jsx
import { Card, CardMedia, CardContent, CardActions, Typography, Button } from '@mui/material';

function BookCard({ titulo, autor, imagen }) {
 return (
   <Card sx={{ maxWidth: 345 }}>
     <CardMedia component="img" height="140" image={imagen} alt={titulo} />
     <CardContent>
       <Typography gutterBottom variant="h5" component="div">
         {titulo}
       </Typography>
       <Typography variant="body2" color="text.secondary">
         {autor}
       </Typography>
     </CardContent>
     <CardActions>
       <Button size="small" variant="contained">Ver Detalles</Button>
     </CardActions>
   </Card>
 );
}
```



Recursos y Referencias

🅱️ Bootstrap

- **Documentación:** react-bootstrap.github.io
- **Componentes:**
  Navbar, Card, Button, Form, Modal
- **Grid System:**
  Container, Row, Col

🟦 Material-UI

- **Documentación:** mui.com
- **Componentes:**
  AppBar, Card, Button, TextField, Dialog
- **Theming:**
  Sistema de colores y tipografía

💡 Recomendación

Elegir **UNA** librería y dominarla



Recursos y Referencias

📖 Documentación Oficial

- React Dev

  Será la fuente principal de seguimiento

Revisar de la documentación oficial la sección: "Aprende React > Inicio rápido > **Pensar en React**"
Tener en cuenta al menos hasta el punto 3 (pueden omitir por ahora los detalles de los Estados)

🛠️ Setup Recomendado

- ⚡ Vite (npm create vite@latest)
- 🛠️ React Developer Tools (extensión Chrome)
- 💻 VS Code Extensions: ES7+ React/Redux/React-Native snippets

Actividades

Actividad Individual de la semana

1° instancia hasta el **24/05/2026**
*2° instancia: **05/07/2026** (Cierre Bloque 2)*

[AI-C08 - Librería en React]

Maquetar la home de la librería en App.tsx

1. En el repositorio ds2026-actividades en GitHub, en un subdirectorio nuevo para esta clase: **ejercicios/c08-libreria-react/** Crear proyecto Vite + React + TS
2. Sumar react-bootstrap (u otra Librería UI)
3. Maquetar con React/TS/Librería UI la home del ejercicio de la librería de la clase 6.
4. Implementar componente reutilizable para la card de libros
5. Crear al menos 2 componentes más propios (ej: **<Navbar />**, **<Footer />** o lo que quieran para una librería).
6. Al menos un componente debe usar **useState** (ej: botón "Me gusta", contador, toggle).

Subir al repositorio y comunicar la finalización en Discord en un mensaje único por equipo, dentro del canal **#actividades**, en el Hilo: "***[AI-C08] Librería en React***".

Actividad grupal

Hasta el **31/05/2026** (inclusive)

[AG-C08 - Setup React + Home]

Maquetar la home del TP grupal en App.tsx:

1. Crear la carpeta "**frontend**" en el repositorio del TP Grupal
2. Adentro, crear proyecto Vite + React + TS.
3. Elegir e instalar react-bootstrap (u otra Librería UI)
4. Identificar **componentes** de la home (en **App.tsx**, todos en el mismo archivo está bien por ahora).
5. Cada integrante crea **al menos 1 componente** con props tipadas, en branch propio + PR al main.
6. Cada compañero revisa/aprueba al menos un PR.
7. Al menos uno de los componentes debe usar `useState`.
8. Trello: mover tarjetas correspondientes a "Done".
9. Comunicar la finalización en Discord en un mensaje único por equipo, dentro del canal **#actividades**, en el Hilo: "***[AG-C08] Setup React + Home***".